Project 3, Part A: Exploring the Power of Flow Models — Student Notebook

Use the current Part A assignment webpage for equations and deliverables. Complete the student functions, run the provided display cells, and present your results on your project webpage. This notebook follows the same three sections:

1. Visualizing Training and Sampling
2. Conditional Generation and Guidance
3. Creative Applications

sdflsjkdflkjsdf

Start here

1. Select a **GPU**: Colab **Runtime → Change runtime type**, or Kaggle **Settings → Accelerator**. Enable Internet on Kaggle.
2. Run the **Setup** cell below. Everything needed is included or downloaded automatically.
3. Complete the exercises below and run their cells.

No ZIP upload, Google Drive mount, helper-file edits, or cache configuration is needed. Use a fresh runtime when switching between Part A and Part B. Downloads are automatically reused within the session. Save your notebook with outputs before disconnecting.

Setup (run once)

Show code

```
GPU: NVIDIA A100-SXM4-40GB VRAM GiB: 39.5
PyTorch: 2.11.0+cu130 torchvision: 0.26.0+cu130
Local cache: /content/cs180-hw3/.cache
Part A precision: float32 (text encoder and image model).
Setup complete. Continue with the notebook below.
```

```
import os, json, time, math
from pathlib import Path
from uuid import uuid4
```

```
from uuid import uuid4
import torch
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from IPython.display import display
from torchvision.transforms.functional import gaussian_blur
from flow_model import FlowModel, preprocess, to_pil, MODEL_ID, MODEL_

torch.set_num_threads(4)
ROOT = Path.cwd()
GROUP = os.environ.get("FLOW_GROUP", "all")
if GROUP not in ("all", "core", "editing", "illusions"):
    raise ValueError("FLOW_GROUP must be all, core, editing, or illus
# A new folder for this run keeps earlier results available.
RUN_NAME = time.strftime("%Y%m%d-%H%M%S") + "-" + uuid4().hex[:6]
OUT = ROOT / "results" / "parta_updated" / RUN_NAME
OUT.mkdir(parents=True, exist_ok=False)
SEED, H, W = 180, 0.02, 7
torch.manual_seed(SEED)
_model_key = (MODEL_REVISION, TEXT_REVISION, 'cuda:0', 'float32')
if globals().get('_loaded_model_key') != _model_key or 'model' not in
    model = FlowModel()
    _loaded_model_key = _model_key
else:
    print('Reusing the already-loaded PixNerd model in this kernel.')
predict_velocity = model.predict_velocity
device = model.device
condition = model.encode("a high quality photo")
null_condition = model.encode("")


def load_input(path):
    if not str(path).strip():
        raise ValueError("Set the image path to a file you uploaded o
    path = Path(path).expanduser()
    if not path.is_file():
        raise FileNotFoundError(f"Image not found: {path}. Check the
    return preprocess(path).to(device)


def noise(seed):
    return torch.randn((1, 3, 512, 512), generator=torch.Generator().m


def panel(name, images, columns=4, title=None):
    items = list(images.items())
    fig, axs = plt.subplots(math.ceil(len(items) / columns), columns,
                            figsize=(4 * columns, 4 * math.ceil(len(i
                            squeeze=False, layout="constrained")
    if title is not None:
```

```
        fig.suptitle(title, wrap=True)
    for ax in axs.flat:
        ax.axis("off")
    for ax, (label, value) in zip(axs.flat, items):
        ax.imshow(to_pil(value))
        ax.set_title(label, fontsize=9, wrap=True)
    fig.savefig(OUT / (name + ".png"), dpi=110)
    plt.show()
    plt.close(fig)


def save(name, x):
    to_pil(x).save(OUT / (name + ".png"))


def guided(x, t, c=condition, w=W):
    return cfg_velocity(x, t, c, w, predict_velocity, null_condition)


metrics = {"group": GROUP, "seed": SEED, "step_size": H, "cfg_w": W,
           "model": MODEL_ID, "revision": MODEL_REVISION, "text_revis
           "gpu": torch.cuda.get_device_name(0), "visible_devices": o
           "torch": torch.__version__}
print("Results folder:", OUT)
print(metrics)
```

```
/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_auth.py
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your sett
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to a
  warnings.warn(
model.ckpt: 100%                                                  4.70G/4.70G [00:14<00:00, 313MB/
                                                                  s]
tokenizer_config.json:        9.73k/? [00:00<00:00, 1.03MB/s]
vocab.json:        2.78M/? [00:00<00:00, 88.3MB/s]
merges.txt:        1.67M/? [00:00<00:00, 76.7MB/s]
tokenizer.json: 100%                                              11.4M/11.4M [00:00<00:00, 28.1MB/
                                                                  s]
config.json: 100%                                                   726/726 [00:00<00:00, 102kB/
                                                                  s]
model.safetensors.index.json:        25.6k/? [00:00<00:00, 3.13MB/s]
Fetching 2 files: 100%                                                2/2 [00:10<00:00, 10.42s/
                                                                  it]
```

```
model-00002-                                622M/622M [00:02<00:00, 494MB/
of-00002.safetensors: 100%                  s]
```

Functions to implement

Run the cell below to define the function stubs. Fill in each function as you reach its section, then rerun this cell after editing it. Follow the equations on the Part A webpage. Image loading, text encoding, model evaluation, rotation, and low-pass filtering are provided. The velocity-visualization exercise has its own student function in Section 1.4.

```
# STUDENT IMPLEMENTATION: complete each function in its corresponding

def interpolate(image, noise, t):
    return t * image + (1 - t) * noise
    # raise NotImplementedError("Implement interpolate in Section 1.1


def one_step_denoise(x, t, condition, predict_velocity):
    with torch.no_grad():
      x1_hat = x + (1-t) * predict_velocity(x, t, condition)
      return x1_hat
    # raise NotImplementedError("Implement one_step_denoise in Sectio


@torch.inference_mode()
def euler_sample(x_start, t_start, step_size, velocity):
    states = [(t_start, x_start)]

    while t_start < 1:
      dt = min(step_size, 1 - t_start)
      x_start = x_start + dt*velocity(x_start, t_start)
      t_start = min(1, t_start + dt)
      states.append((t_start, x_start))


    return x_start, states

    # raise NotImplementedError("Implement euler_sample in Section 1.


def cfg_velocity(x, t, condition, w, predict_velocity, null_condition
    v_uncond = predict_velocity(x, t, null_condition)
    v_cond = predict_velocity(x, t, condition)

    return v_cond + w*(v_cond - v_uncond)
```

```
    # raise NotImplementedError("Implement cfg_velocity in Section 2.1
```

```
def edit(image, noise, t_start, condition, predict_velocity, w=7, step
    inverted = t_start * image + (1 - t_start) * noise
    return euler_sample(inverted, t_start, step_size, lambda x, t: gu
    # raise NotImplementedError("Implement edit in Section 3.1.")
```

```
# PROVIDED: rotate by 180 degrees, keeping the batch and RGB axes uncl
def rotate180(x):
    return torch.flip(x, (-2, -1))
```

```
def visual_anagrams(noise, condition_a, condition_b, predict_velocity
    def avg_v_helper(x, t):
      v1 = cfg_velocity(x, t, condition_a, w, predict_velocity)
      v2 = rotate180(cfg_velocity(rotate180(x), t, condition_b, w, pr
      return 0.5 * (v1 + v2)
    return euler_sample(noise, 0.0, step_size, avg_v_helper)[0]
    # raise NotImplementedError("Implement visual_anagrams in Section
```

```
# PROVIDED: at 512 x 512, this uses kernel size 265 and sigma 16.
def lowpass(x, sigma=16.0):
    kernel = 2 * round(16.5 * x.shape[-1] / 64) + 1
    return gaussian_blur(x, [kernel, kernel], [sigma, sigma])

def highpass(x):
    lowpass_img = lowpass(x)
    return x - lowpass_img
```

```
def make_hybrids(noise, condition_a, condition_b, predict_velocity, w=
    def factorized_helper(x, t):
      v1 = lowpass(cfg_velocity(x, t, condition_a, w, predict_velocity
      v2 = highpass(cfg_velocity(x, t, condition_b, w, predict_velocity
      return (v1 + v2)
    return euler_sample(noise, 0.0, step_size, factorized_helper)[0]
    # raise NotImplementedError("Implement make_hybrids in Section 3.4
```

1. Visualizing Training and Sampling

Use the linear flow-matching convention from the webpage: $x_t = (1-t)\epsilon + tx_1$, where $\epsilon \sim \mathcal{N}(0, I)$. **Time 0 is noise; time 1 is a clean image.** The path velocity target is $x_1 - \epsilon$. We use the pretrained model; no model training is required in Part A.

Choose your image for Sections 1.1–1.5

Use **one image of your own, not the provided image of the Campanile**, throughout these five exercises. Upload or add your image to the runtime, then set `YOUR_IMAGE_PATH` below to its path. In Colab, upload through the Files sidebar and copy the uploaded file's path. In Kaggle, add the image as an input and copy its path from the file browser.

The supplied preprocessing center-crops and resizes the image to 512×512 and normalizes RGB to $[-1, 1]$. The Campanile will be loaded separately for Sections 3.1–3.2.

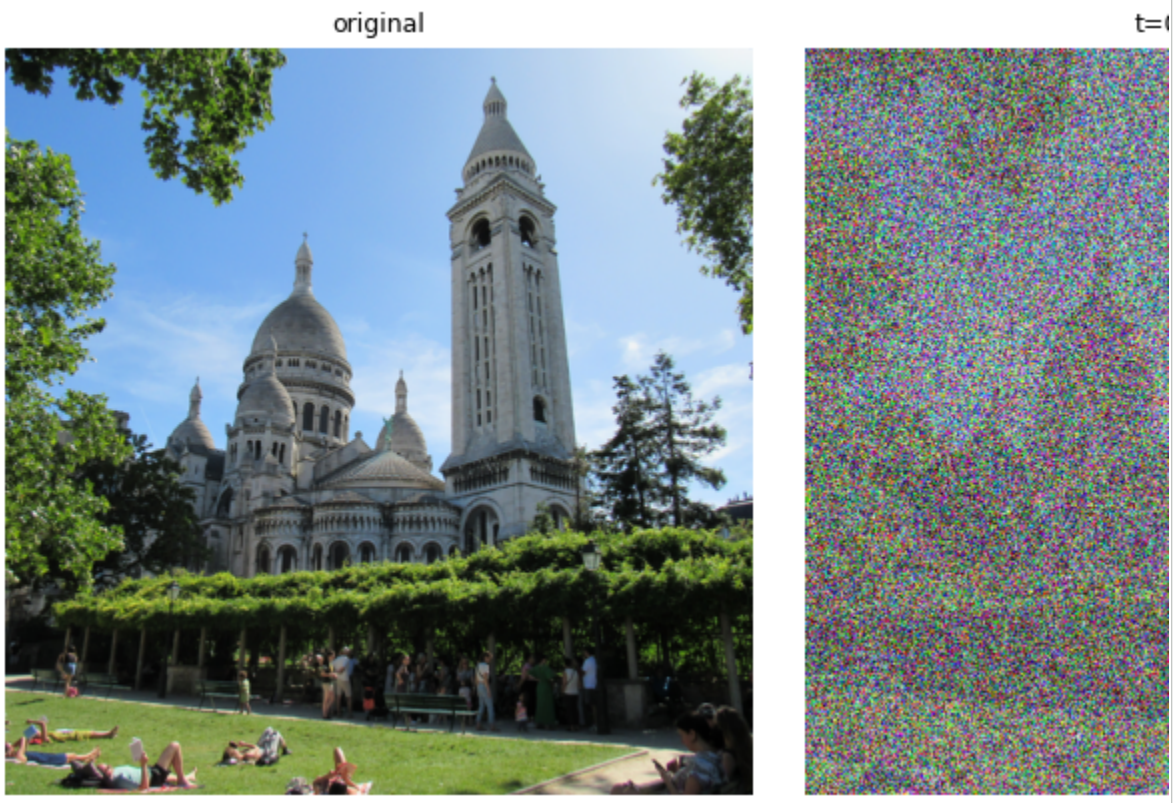
```
YOUR_IMAGE_PATH = "/content/cs180-hw3/assets/montmartre.JPG"
image = load_input(YOUR_IMAGE_PATH)
metrics["own_image_path"] = str(Path(YOUR_IMAGE_PATH).expanduser().resc
display(to_pil(image))
```

1.1 Constructing Noisy Training Inputs

Implement `interpolate(image, noise, t)` using $x_t = (1-t)\epsilon + tx_1$. Sample one noise tensor and reuse it at $t = 0.25, 0.50, 0.75$. Show your original image and the three noisy inputs, labeled by time.

Hints: Check both endpoints. Keep `image`, `epsilon`, and `noisy` for Sections 1.2–1.5; the original image/noise pair is needed to compute the ground-truth velocity in Section 1.4.

```
epsilon = noise(SEED)
noisy = {t: interpolate(image, epsilon, t) for t in [.25,.50,.75]}
assert torch.equal(interpolate(image,epsilon,0),epsilon)
assert torch.equal(interpolate(image,epsilon,1),image)
if GROUP in ("all","core"):
    torch.save({"image": image.cpu(), "epsilon": epsilon.cpu(),
                "noisy": {t: x.cpu() for t, x in noisy.items()}}, OUT /
    panel("training_inputs", {"original": image, **{f"t={t}":x for t,x
```

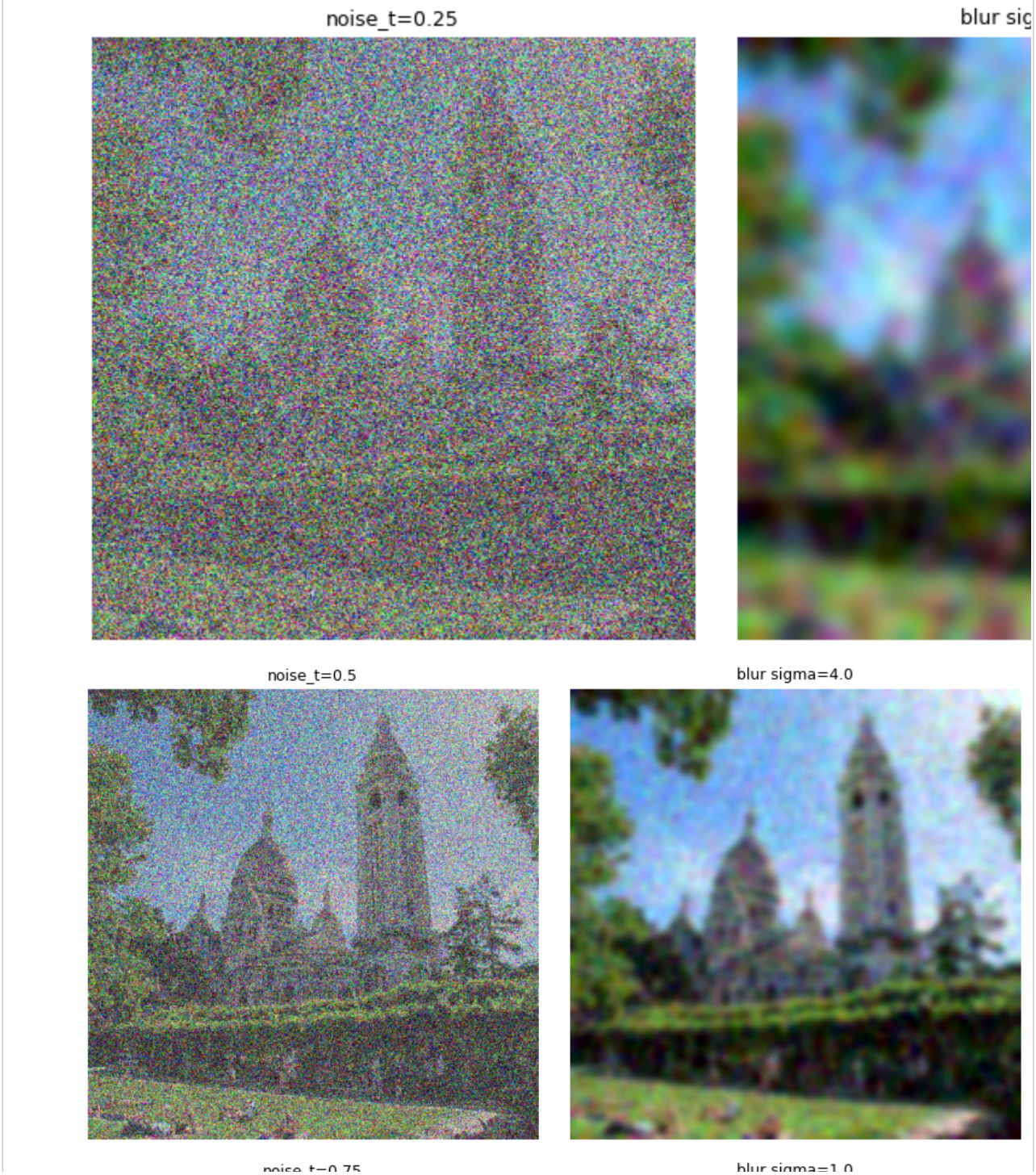


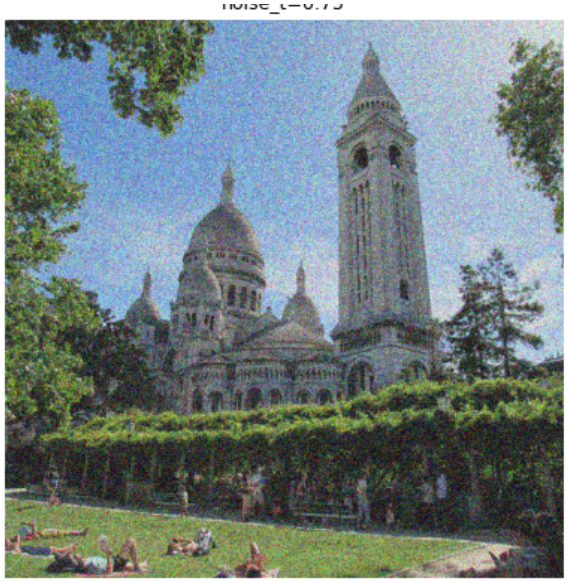
1.2 Classical Denoising

For each saved input, restore the clean-image amplitude with x/t, then apply a Gaussian filter. Try different blur strengths and show your best result beside each noisy input. The provided cell sweeps sigma=1,2,4,8,16 and selects lowest mean

squared error to the known clean image; this is a baseline comparison, not a learned model or a guarantee of perceptual quality.

```
if GROUP in ("all","core"):
    blurred = {}
    for t,x in noisy.items():
        candidates = {s: gaussian_blur(x/t, [int(6*s+1)]*2, [s,s]) for
        sigma = min(candidates, key=lambda s: ((candidates[s]-image)**2
        blurred[t] = candidates[sigma]
        panel(f"blur_{t}", {f"noise_t={t}":x,f"blur sigma={sigma}":blur
```



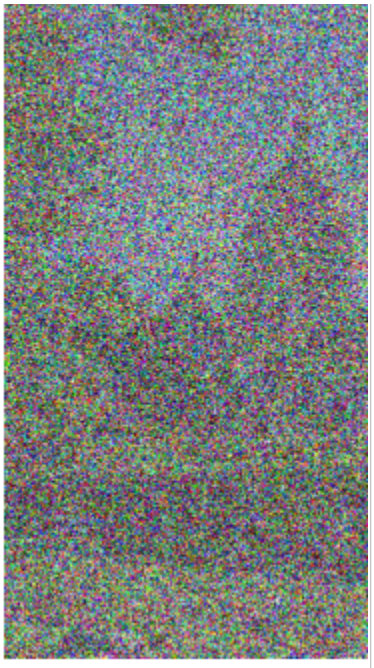


1.3 One-Step Denoising

Implement `one_step_denoise`: $\hat{x}_1 = x_t + (1-t)v_\theta(x_t, t, c)$. Use exactly one prediction for each of the same three noisy tensors, with "a high quality photo". Display original, input, and estimate. Check velocity shape, dtype, and device.

```
if GROUP in ("all","core"):
    v = predict_velocity(noisy[.5], .5, condition)
    assert v.shape == image.shape and v.dtype == image.dtype and v.devi
    assert torch.isfinite(v).all()
    one_step = {t:one_step_denoise(x,t,condition,predict_velocity) for
    for t,x in noisy.items():
        panel(f"one_step_{t}", {"original":image,f"noise_t={t}":x,"one
```

```
/content/cs180-hw3/vendor/PixNerd/src/models/transformer/pixnerd_t2i.p
  pos = pos[None, :, :].to(device=device, dtype=dtype)
```



original

noise

original

noise_t=0.75

one step

1.4 Visualizing Velocity

In Section 1.3, we added $(1-t)\hat{v}_t$ to the noisy image to estimate the clean image. Here, visualize what the model adds and compare it with the update $x_1 - x_t$ that would recover your original image.

Use the same image and saved noisy inputs at $t = 0.25, 0.50, 0.75$, with `"a high quality photo"`.

Deliverables

- Show the original image once.
- At each time, show the noisy input, predicted update $(1-t)\hat{v}_t$, ground-truth update $(1-t)v_{\text{gt}}$, and an error heatmap, labeled by time. You may arrange the displays however you like.

Implementation hints

- Predict $\hat{v}_t = v_\theta(x_t, t, c)$. Using the saved noise, compute $v_{\text{gt}} = x_1 - \epsilon$, so $(1-t)v_{\text{gt}} = x_1 - x_t$.
- For the error map, use RMS across RGB **at each pixel**: $E_t = \sqrt{\text{mean}_{\text{RGB}}([(1-t)(\hat{v}_t - v_{\text{gt}})]^2)}$. Square the channel differences, average over RGB, and take the square root before display scaling.
- Use one shared symmetric RGB scale for all updates, with zero at middle gray.
- Use `matplotlib.pyplot.imshow` with a linear scale for errors. Choose `vmin` and `vmax` that work for your image, keeping them the same at all three times.

Complete the calculation function below. The following cell supplies an example display.

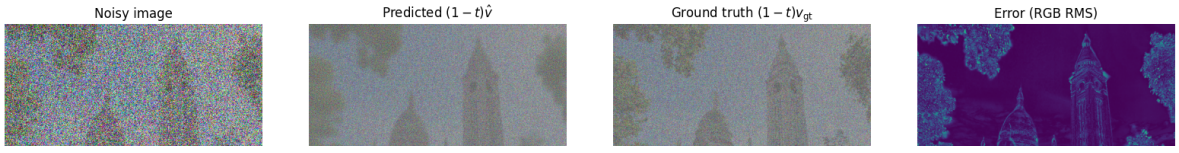
```
@torch.inference_mode()
```

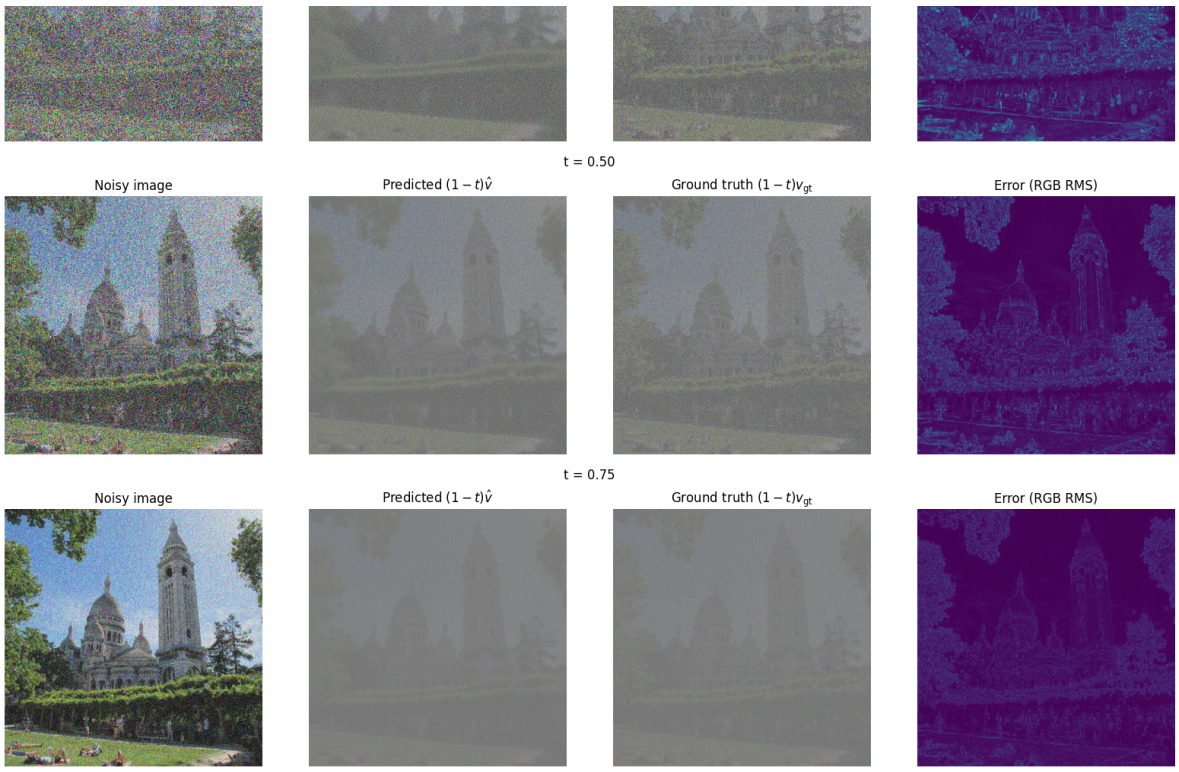
```
def velocity_diagnostics(x_t, t, clean_image, epsilon, condition, pre
    """Return predicted_update, ground_truth_update, and error_map.

    Both updates have shape (B, 3, H, W).
    The error map has shape (B, H, W): average only over RGB, keeping
    Return raw values before display scaling or clipping.
    """
    # STUDENT IMPLEMENTATION: use the equations in Section 1.4.
    v_hat = predict_velocity(x_t, t, condition)
    v_gt = clean_image - epsilon
    pred_update = (1-t)*v_hat
    gt_update = (1-t)*v_gt

    error_map = torch.sqrt(torch.mean(torch.square(pred_update - gt_u

    return pred_update, gt_update, error_map
    # raise NotImplementedError("Compute the two updates and their pe
```

```
# PROVIDED display helpers. Adjust the error limits or layout for you
ERROR_VMIN = 0.0
ERROR_VMAX = None  # None uses the maximum across all three maps; you


def signed_rgb(update, limit):
    # Map the shared signed range [-limit, limit] to RGB [0, 1].
    value = update.detach().float().cpu()[0].permute(1, 2, 0)
    return (0.5 + value / (2 * limit)).numpy()


if GROUP in ("all", "core"):
    velocity_results = {}
    for t in (0.25, 0.50, 0.75):
        predicted_update, target_update, error_map = velocity_diagnos
            noisy[t], t, image, epsilon, condition, predict_velocity)
        if predicted_update.shape != image.shape or target_update.sha
            raise ValueError("Each update should have the same shape a
        if error_map.shape != (image.shape[0], *image.shape[-2:]):
            raise ValueError("The error map should have shape (B, H, W
        velocity_results[t] = tuple(value.detach().float().cpu() for v
                                    (predicted_update, target_update,

    signed_limit = max(1e-8, *(float(value.abs().max())
                               for predicted, target, _ in velocity_res
                               for value in (predicted, target)))
    error_max = ERROR_VMAX
    if error_max is None:
        error_max = max(1e-8, *(float(error.max()) for _, _, error in
    if error_max <= ERROR_VMIN:
        raise ValueError("Choose ERROR_VMAX greater than ERROR_VMIN."
```

```
print("Original image")
display(to_pil(image))
save("velocity_original", image)
for t, (predicted_update, target_update, error_map) in velocity_r
    fig, axes = plt.subplots(1, 4, figsize=(16, 4), layout="const
    fig.suptitle(f"t = {t:.2f}")
    axes[0].imshow(to_pil(noisy[t]))
    axes[1].imshow(signed_rgb(predicted_update, signed_limit))
    axes[2].imshow(signed_rgb(target_update, signed_limit))
    axes[3].imshow(error_map[0].numpy(), vmin=ERROR_VMIN, vmax=er
    for ax, title in zip(axes, ("Noisy image", r"Predicted $(1-t)
                                r"Ground truth $(1-t)v_{\mathrm{g
        ax.set_title(title)
        ax.axis("off")
    fig.savefig(OUT / f"velocity_{t:.2f}.png", dpi=110)
    plt.show()
    plt.close(fig)
```

```
Original image
```





1.5 Iterative Denoising with Euler Sampling

Implement `euler_sample(x_start,t_start,step_size,velocity)`: $x_{t+h}=x_t+h\cdot v(x_t,t)$. Recompute velocity each step and shorten the final step to reach 1 exactly. Return the final tensor and saved states. Denoise the saved t=0.50 input with

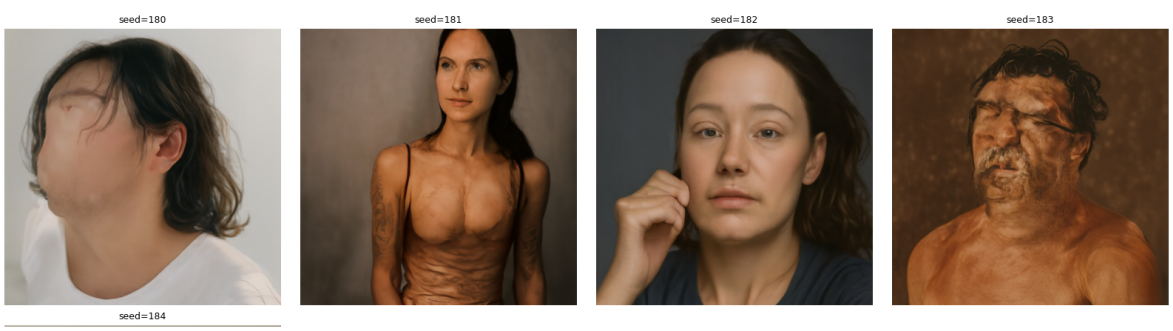
exactly. Return the final tensor and saved states. Denoise the saved t=0.50 input with h=0.02 (25 updates). Show the initial state, every fifth update, and final result, labeled by time. Compare blur, one-step, and iterative results on this same input.

```
if GROUP in ("all","core"):
    calls=[]
    def constant_v(x,t):
        calls.append(t); return torch.ones_like(x)
    toy=torch.zeros(1,3,4,4,device=device)
    for start,step,expected in [(0,.02,50),(.5,.02,25),(.93,.02,4)]:
        calls.clear(); end,states=euler_sample(toy,start,step,constant_
        assert len(calls)==expected and states[-1][0]==1
        assert torch.allclose(end,torch.full_like(toy,1-start),atol=1e-
    iterative,trajectory=euler_sample(noisy[.5],.5,H,lambda x,t:predict
    panel("trajectory",{f"t={t:.2f}":x for t,x in trajectory[::5]},3)
    panel("comparison",{"original":image,"blur":blurred[.5],"one step":
    metrics["reconstruction_mse"]={k:float(((v-image)**2).mean()) for k
        {"blur":blurred[.5],"one_step":one_step[.5],"euler":iterative}.
    metrics["euler_checks"]="50 full steps, 25 half steps, shortened fi
```

1.6 Generating Images from Noise

Generate five independent Gaussian initial states and integrate from 0 to 1 (50 Euler updates). Use “a high quality photo”. Show all five results and record the seeds.

```
if GROUP in ("all","core"):
    samples={}
    for i in range(5):
        x,_=euler_sample(noise(SEED+i),0,H,lambda x,t:predict_velocity(
        samples[f"seed={SEED+i}"]=x; save(f"conditional_{i}",x)
    panel("conditional_samples",samples)
```



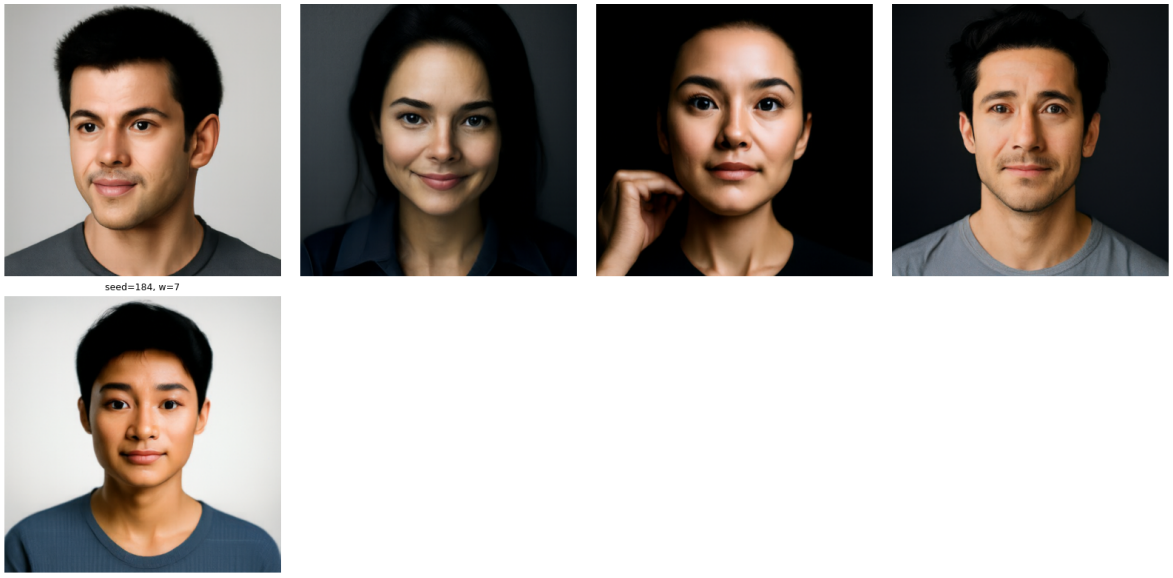
2. Conditional Generation and Guidance

Section 1 used a fixed text prompt. Here, implement classifier-free guidance to strengthen the prompt's effect and compare the resulting samples.

2.1 Classifier-Free Guidance

Implement v_cfg=v_c+w(v_c−v_u), with empty-string null condition. Both branches must evaluate the same x and t. w=0 gives ordinary conditional prediction. Generate five images at w=7 and h=0.02. Compare to Section 1.6 using the same initial noises.

```
if GROUP in ("all","core"):
    x=noise(SEED)
    assert torch.equal(cfg_velocity(x,.5,condition,0,predict_velocity,r
    cfg_samples={}
    for i in range(5):
        x,_=euler_sample(noise(SEED+i),0,H,guided)
        cfg_samples[f"seed={SEED+i}, w={W}"]=x; save(f"cfg_{i}",x)
    panel("cfg_samples",cfg_samples)
    metrics["cfg_zero_check"]=True
```



3. Creative Applications

Use Euler step size `h=0.02`, CFG weight `w=7`, and the empty-string prompt for the unconditional prediction throughout this section.

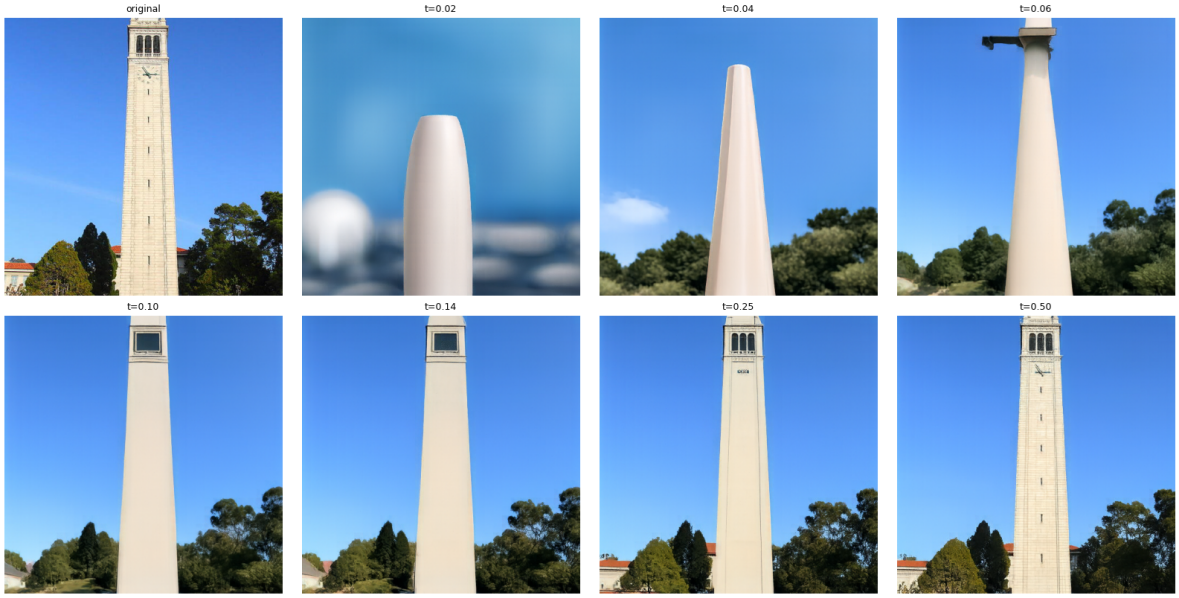
3.1 Image-to-Image Editing

Implement `edit`: construct $(1 - t_{\text{start}})\epsilon + t_{\text{start}} x_1$ and integrate from the starting time to 1 with CFG. Use `"a high quality photo"` and the **seven** starting times **0.02, 0.04, 0.06, 0.10, 0.14, 0.25, 0.50**.

Show the original and all seven edits for the **Campanile and two images of your own**. The Campanile is included in setup. Set the two image paths below; you may reuse your image from Section 1.1 as one of them. These inputs will also be used in

Section 3.2.

```
# The same seven times are used throughout Sections 3.1 and 3.2.
STARTS = [0.02, 0.04, 0.06, 0.10, 0.14, 0.25, 0.50]
TEST_IMAGE_PATHS = {
    "my_image_1": "/content/cs180-hw3/assets/montmartre.JPG",
    "my_image_2": "/content/cs180-hw3/assets/yofi.JPG",  # Add the path
}

if GROUP in ("all", "editing"):
    # Load Campanile separately; `image` remains your Section 1 image.
    campanile = load_input(ROOT / "assets/campanile.jpg")
    edit_inputs = {"campanile": campanile,
                   **{name: load_input(path) for name, path in TEST_IMA
    for name, y in edit_inputs.items():
        panel("edit_" + name, {"original": y, **{
            f"t={t:.2f}": edit(y, noise(SEED), t, condition, predict_ve
            for t in STARTS}})
        print("Edited", name, flush=True)
```



Edited campanile

Edited my_image_1

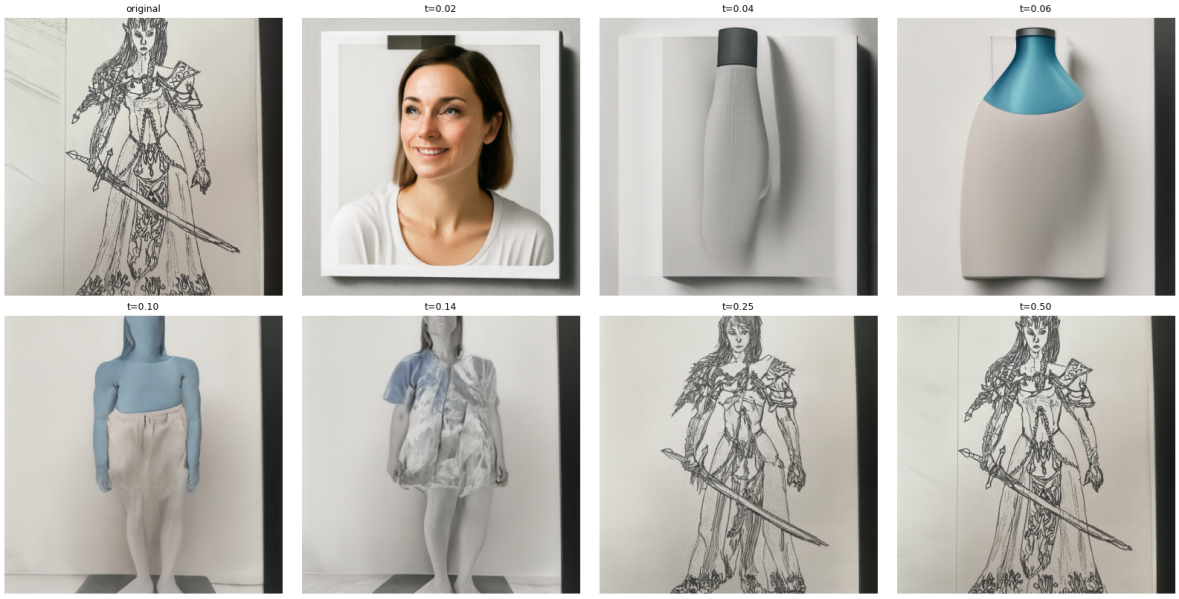
Edited my_image_2

Editing Hand-Drawn and Web Images

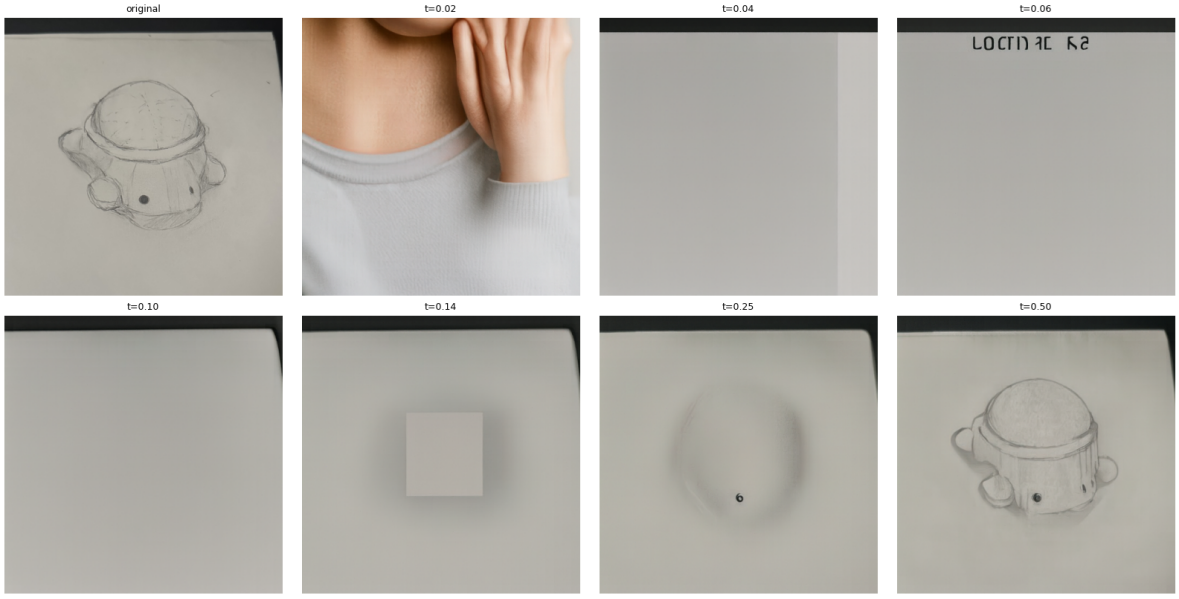
Repeat the same procedure and **seven starting times** for **one image from the web and two of your own drawings**. Download or draw the images, upload/add them to the runtime, and set their paths below. The preprocessing helper center-crops to a square and resizes to 512×512, so keep your drawings centered.

```
WEB_IMAGE_PATH = "/content/cs180-hw3/assets/porter.png"  # Path of an i
DRAWING_PATHS = {
    "drawing_1": "/content/cs180-hw3/assets/zelda.jpeg",  # Path of you
    "drawing_2": "/content/cs180-hw3/assets/egg.jpeg",  # Path of your
}

if GROUP in ("all", "editing"):
    creative_inputs = {"web_image": load_input(WEB_IMAGE_PATH),
                       **{name: load_input(path) for name, path in DRAW
    for name, y in creative_inputs.items():
        panel("edit_" + name, {"original": y, **{
            f"t={t:.2f}": edit(y, noise(SEED), t, condition, predict_ve
            for t in STARTS}})
        print("Edited", name, flush=True)
```

Edited web_image



Edited drawing_1



Edited drawing_2

3.2 Text-Guided Image Editing

Reuse the Campanile and two personal images from Section 3.1. For each image, choose a prompt describing your desired edit and replace the generic photo prompt with its embedding. Use the same **seven starting times: 0.02, 0.04, 0.06, 0.10, 0.14, 0.25, 0.50**. Show the original and all edits, with the chosen prompt and times.

Larger starting times preserve more of the original image. Smaller starting times add more noise and give the model more room to change the image according to the prompt.

```
    EDIT_PROMPTS = {
```

```
    "campanile": "a photograph of gothic bell tower oozing blue and g
    "my_image_1": "a photo of a pool party in the park and fireworks
    "my_image_2": "an old detective dog with glasses, a top hat, and
}

if GROUP in ("all", "editing"):
    if set(EDIT_PROMPTS) != set(edit_inputs):
        raise ValueError("Provide one editing prompt for each image f
    if any(not prompt.strip() for prompt in EDIT_PROMPTS.values()):
        raise ValueError("Fill in all three EDIT_PROMPTS before runni
    for name, prompt in EDIT_PROMPTS.items():
        c = model.encode(prompt)
        y = edit_inputs[name]
        panel("text_edit_" + name, {"original": y, **{
            f"t={t:.2f}": edit(y, noise(SEED), t, c, predict_velocity
            for t in STARTS}}, title=f'{name}: "{prompt}"')
        print("Text edited", name, flush=True)
```

```
Text edited campanile
```

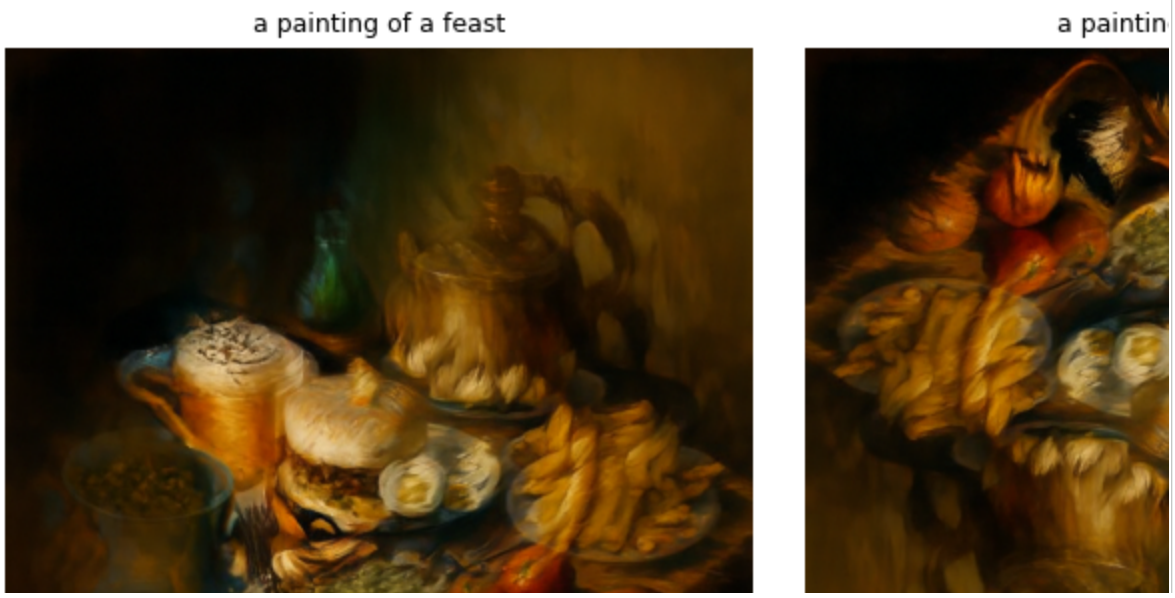
Text edited my_image_1

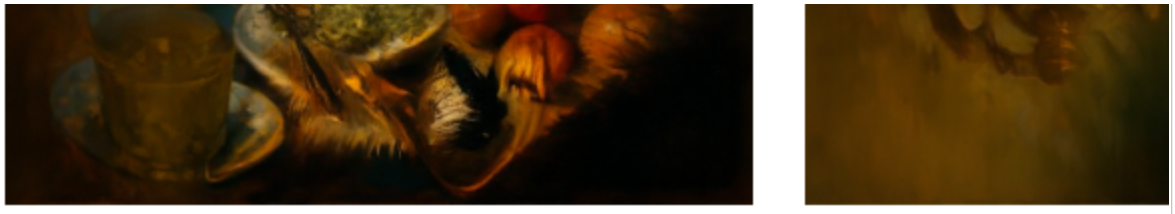
Text edited my_image_2

3.3 Visual Anagrams

Let F rotate an image 180 degrees (flip both spatial axes, not just vertical reflection). Average v_cfg(x,t,c_a) and F(v_cfg(F(x),t,c_b)), and integrate using the same Euler sampler. Choose two prompt pairs and generate two illusions. Show each upright and rotated with both prompts. Several prompt/seed attempts may be needed: a successful numerical run does not imply both subjects are recognizable.

```
if GROUP in ("all","illusions"):
    assert torch.equal(rotate180(rotate180(image)),image)
    # Example prompt pairs; edit these to choose your two illusions.
    pairs=[("a painting of a feast", "a painting of a wolf"),("a painti
           ]
    for i,(a,b) in enumerate(pairs):
        x=visual_anagrams(noise(SEED+i),model.encode(a),model.encode(b)
        save(f"anagram_{i}",x); panel(f"anagram_views_{i}",{a:x,b:rotat
    metrics["rotation_involution_check"]=True
```





a painting of a capybara

a painting of pineapples

3.4 Hybrid Images

Implement `make_hybrids` using `lowpass(v_a) + v_b - lowpass(v_b)` in each Euler update. Prompt `a` determines the distant structure; prompt `b` determines the close-up detail. Use the same image and time for both CFG predictions. The provided low-pass helper uses Gaussian blur with sigma 16 and kernel size 265 at 512×512.

Choose two prompt pairs and generate **two hybrid images**. Show each at full and reduced size, labeling both prompts. The example pairs below match the subjects and viewing directions shown on the webpage; you can choose other pairs. Several attempts may be needed.

```
if GROUP in ("all", "illusions"):
    # (distant / low-frequency prompt, close / high-frequency prompt)
    pairs = [("a painting of a boot", "a painting of a soldier"),
             ("a painting of a fish", "a painting of a house")]
```

```
    for i, (distant_prompt, close_prompt) in enumerate(pairs):
        x = make_hybrids(noise(SEED+i), model.encode(distant_prompt), m
                         predict_velocity, w=W, step_size=H)
        save(f"hybrid_{i}", x)
        print("Close:", close_prompt)
        display(to_pil(x))
        print("Distant:", distant_prompt)
        display(to_pil(x).resize((64, 64), Image.Resampling.LANCZOS))
    metrics["hybrid_lowpass_sigma"] = 16
```

```
Close: a painting of a soldier
```

```
Distant: a painting of a boot
```

```
Close: a painting of a house
```

```
Distant: a painting of a fish
```

Bells & Whistles

Required for CS280A students:

- Create visual anagrams using two additional transformations from the Visual Anagrams paper.
- Design a course logo, for example by applying text-guided image editing to a drawing or the UCB logo.

Optional for everyone: Explore your own ideas. Add cells below for your experiments and include the results on your project webpage.

```
# Add your Bells & Whistles experiments here.
```

Save your results

Inspect your saved images as well as the numerical checks. The cell below saves the run's settings and metrics alongside its figures. Save this notebook with its outputs before disconnecting from the runtime.

```
metrics["peak_memory_gib"]=torch.cuda.max_memory_allocated()/2**30
(OUT/"metrics.json").write_text(json.dumps(metrics,indent=2))
print(json.dumps(metrics,indent=2))
```


```
{
  "group": "all",
  "seed": 180,
  "step_size": 0.02,
  "cfg_w": 7,
  "model": "MCG-NJU/PixNerd-XXL-P16-T2I",
  "revision": "9fa8836b553976b88a3b27649ed1628499e02bf9",
  "text_revision": "70d244cc86ccca08cf5af4e1e306ecf908b1ad5e",
  "gpu": "NVIDIA A100-SXM4-40GB",
  "visible_devices": null,
  "torch": "2.11.0+cu130",
  "own_image_path": "/content/cs180-hw3/assets/montmartre.JPG",
  "rotation_involution_check": true,
  "hybrid_lowpass_sigma": 16,
  "peak_memory_gib": 11.942351818084717
}
```


Deliverable Checklist

- Include every section's required results and explanations on your project webpage.
- Submit your website PDF and code to the corresponding Gradescope assignments.
- The Google Form is not required for Part A; complete it after both parts are finished.
- CS280A students should also include both required Bells & Whistles tasks.

The project is due October 20, 2026. Follow the assignment webpage for complete submission instructions.